Task 3: Implementation Patterns

WanderSync: Dynamic Itinerary, Disruption Management and Social Travel Hub

Team 15

Overview

Two design patterns are applied in WanderSync to address recurring structural problems that cannot be resolved by architectural tactics alone. Each pattern is documented with its category, the specific problem it solves within the system, the classes and modules involved, and UML diagrams showing the structure and runtime behaviour.

Pattern 1: Observer

Category: Behavioral

NFRs addressed: NFR2 (Reliability – disruption notification within 10 s); NFR5 (Modifiability – adding a new reaction type requires no changes to the detection service)

Problem

When the Disruption Detection Service identifies a flight delay or cancellation, multiple independent subsystems must react: the Notification Service must alert the user, the Itinerary Management Service must flag affected segments, and the Social Synchronisation Service may need to notify a travel companion. Without a formal decoupling mechanism, the Disruption Detection Service would hold direct references to all three consumers, creating tight coupling. Every new reaction type (e.g., a future re-booking suggestion engine) would require modifying the detection service itself, violating the Modifiability NFR and GRASP Low Coupling.

Solution and Class Mapping

The Observer (Publish-Subscribe) pattern decouples the event source from its consumers.

- `DisruptionEvent` – data object carrying `flightId`, `DisruptionType`, and `Instant` timestamp.
- `IDisruptionObserver` – interface declaring `onDisruption(event: DisruptionEvent)`. All consumers implement this interface.
- `DisruptionEventPublisher` – subject; holds a list of registered `IDisruptionObserver` instances; calls `notifyObservers(event)` after persisting the event to the Redis Stream (Tactic 4).
- `NotificationObserver` – sends an in-app or email notification to the affected user.
- `ItineraryFlagObserver` – marks the affected `BookingRecord` segments as disrupted and invalidates the itinerary cache entry.
- `SocialAlertObserver` – queries the consent registry and notifies matched travel companions who have bilateral opt-in with the affected user.

Adding a new observer (e.g., a re-booking engine) requires implementing `IDisruptionObserver` and registering it at startup – zero changes to `DisruptionEventPublisher`.

Class Diagram

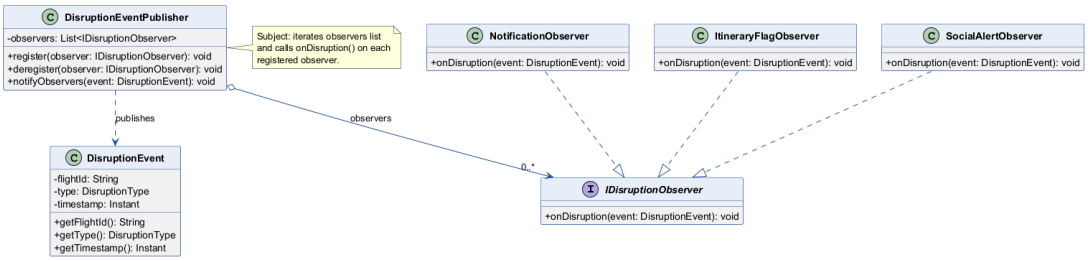

Figure 1: Observer Pattern – Disruption Event Publication

Pattern 2: Chain of Responsibility

Category: Behavioral

NFRs addressed: NFR3 (Security – 100% of endpoints require JWT enforcement, HTTPS); NFR5 (Modifiability – handlers are independently testable and reorderable)

Problem

Every inbound request to the API Gateway must pass through multiple independent validation steps in a defined order: rate-limit check, JWT authentication, request schema validation, and audit logging. Encoding all these steps as a flat conditional block in a single gateway controller method produces a God Method that violates High Cohesion (GRASP) and makes each step impossible to test in isolation. Furthermore, steps must be reorderable (e.g., placing the audit logger after auth so that only authenticated requests are logged) and individually enable/disable-able in test environments.

Solution and Class Mapping

The Chain of Responsibility pattern organises each processing step as an independent handler that either processes the request and passes it to the next handler or short-circuits with an error response.

- `RequestHandler` – abstract base class declaring `setNext(handler)`, the protected `next` reference, and the abstract `handle(request: ApiRequest): ApiResponse` method.
- `RateLimitHandler` – checks the caller's request count against a Redis counter; returns `429 Too Many Requests` if exceeded.
- `AuthHandler` – validates the JWT signature and 24-hour expiry; returns `401 Unauthorized` if invalid, satisfying NFR3.
- `SchemaValidationHandler` – validates the request body against the declared schema; returns `400 Bad Request` on violation.

- `AuditLogHandler` – writes request metadata (user ID, endpoint, timestamp) to the audit log; always passes through to the next handler.
- `GatewayController` – assembles the chain at startup: `RateLimitHandler` → `AuthHandler` → `SchemaValidationHandler` → `AuditLogHandler` → `route dispatch`. Only this class changes when the chain order is modified.

Class Diagram

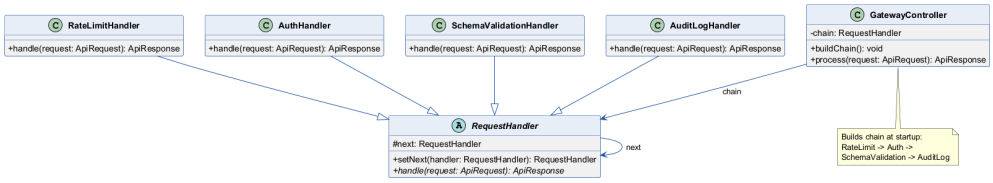

Figure 2: Chain of Responsibility – API Gateway Handler Hierarchy

Sequence Diagram

The sequence diagram below shows a successful request traversing the full chain (happy path) and a short-circuit at `AuthHandler` for an invalid JWT (alternate path).

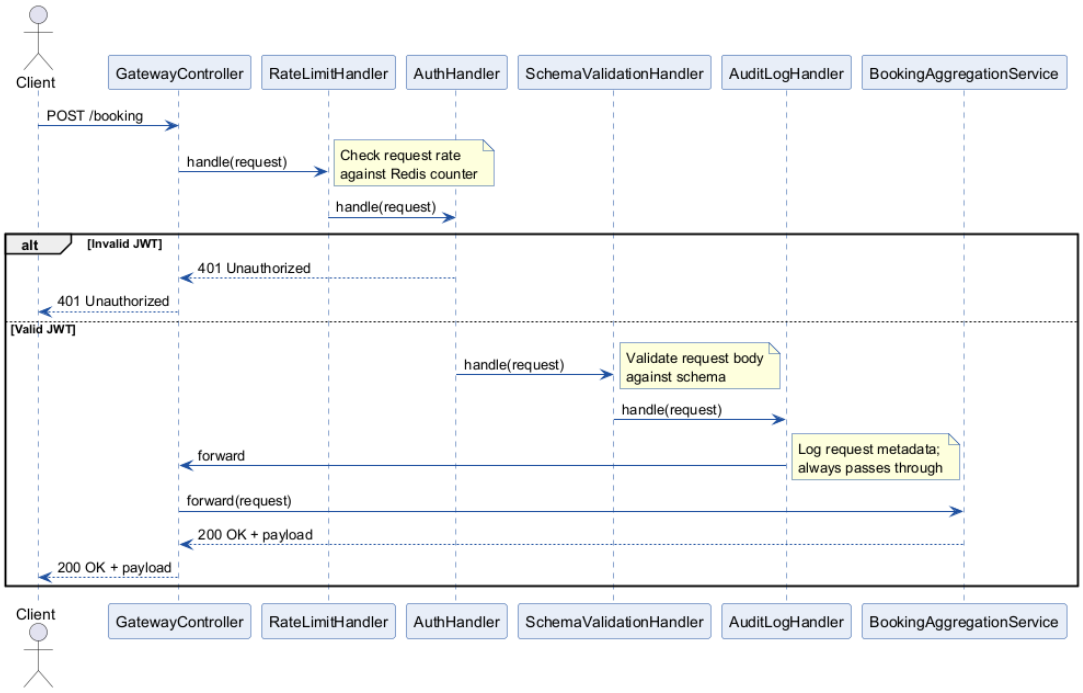

Figure 3: Chain of Responsibility – Request Handling Sequence (happy path and JWT rejection)

Pattern Summary

Pattern	Category	NFRs	Key Classes
Observer	Behavioral	NFR2, NFR5	`DisruptionEventPublisher`, `IDisruptionObserver`, `NotificationObserver`, `ItineraryFlagObserver`, `SocialAlertObserver`
Chain of Responsibility	Behavioral	NFR3, NFR5	`RequestHandler`, `RateLimitHandler`, `AuthHandler`, `SchemaValidationHandler`, `AuditLogHandler`, `GatewayController`